

SISTEMA EMPREL

GUIA DE IMPLANTACAO EM REDE LOCAL

Arquitetura: 1 Servidor + 5 Clientes

Plataforma: Windows

Banco de Dados: PostgreSQL

Versao: 1.0

Data: Junho de 2026

Sumario

1. Diagnostico da Estrutura Atual
2. Por Que Arquivos JSON Compartilhados Nao Funcionam
3. Arquitetura Recomendada
4. Instalacao do PostgreSQL no Servidor
5. Configuracao do Acesso Remoto
6. Compartilhamento da Pasta do Sistema (SMB)
7. Configuracao dos Clientes
8. Migracao dos Dados Existentes (JSON para PostgreSQL)
9. Atalhos e Autostart nas Maquinas Clientes
10. Backups Automaticos
11. Seguranca e Manutencao

1. Diagnostico da Estrutura Atual

1.1 Funcionamento Atual

O Sistema EMPREL funciona atualmente em ambiente monousuario ou compartilhado simples, utilizando arquivos JSON como base de dados local. Os principais arquivos sao:

```
usuarios.json organizacoes.json acoes_excluidas.json  
cadastros_excluidos.json backup_historico.json entregas_confirmadas.json  
servicos_excluidos.json lista_entrega_temp.json janela_config.json db.json
```

1.2 Riscos do Compartilhamento Simples

Quando a pasta do sistema e compartilhada via rede simples (SMB sem controle transacional), os seguintes riscos surgem:

- Corrupção de arquivos JSON quando dois usuarios gravam ao mesmo tempo.
- Perda de dados por conflitos de escrita (write-write conflict).
- Lentidao por trava de arquivo (file lock) imprevisível.
- Dificuldade de auditoria e rastreabilidade.
- Impossibilidade de consultas relacionais (ex: listar acoes por organizacao).

2. Por Que Arquivos JSON Compartilhados Nao Funcionam

2.1 Race Condition

Race condition acontece quando dois ou mais processos tentam modificar o mesmo arquivo simultaneamente. No caso do JSON:

```
Usuario A --> Leitura do arquivo --> Processamento --> Gravacao
Usuario B --> Leitura do arquivo --> Processamento --> Gravacao (sobrescreve A)
```

Nao ha garantia de que B leu os dados ja modificados por A. O ultimo a gravar vence, apagando alteracoes do outro usuario.

2.2 Corrupção de Estrutura

Se a gravacao for interrompida (queda de energia, crash do Windows, fechamento abrupto), o arquivo JSON pode ficar incompleto ou malformado, impedindo o sistema de abrir.

2.3 Ausencia de ACID

JSON nao oferece atomicidade, consistencia, isolamento nem durabilidade. Um banco de dados relacional como PostgreSQL garante essas propriedades, permitindo que multiplos usuarios trabalhem simultaneamente sem corromper os dados.

3. Arquitetura Recomendada

3.1 Visao Geral

A arquitetura recomendada divide as responsabilidades entre o servidor e os clientes:

```
+-----+ +-----+ | SERVIDOR | |
CLIENTE 1 | | - PostgreSQL (banco) | | - Python | | - Pasta compartilhada |
+-----+ | (executaveis + logs) |
+-----+ | - Backups (pg_dump) | | CLIENTE 2..5 |
+-----+ +-----+ | rede TCP/IP 5432
| +-----+
```

3.2 Requisitos de Hardware

Servidor: Computador com Windows 10/11 Pro ou Server, 8 GB RAM, 256 GB SSD, conectado via cabo Ethernet (recomendado).

Clientes: Computadores com Windows 10/11, 4 GB RAM, acesso a rede local.

4. Instalacao do PostgreSQL no Servidor

4.1 Pre-requisitos

- Windows 10/11 Pro, Enterprise ou Windows Server 2016+.
- Usuario com privilégios administrativos.
- Espaço em disco minimo: 2 GB para instalacao + dados.
- Porta 5432 liberada no firewall do servidor.

4.2 Download e Instalacao

Acesse o site oficial e baixe o instalador para Windows:

```
https://www.postgresql.org/download/windows/
```

Ou utilize o instalador via Chocolatey (se disponível):

```
choco install postgresql --params '"--ServerPort 5432"'
```

4.3 Durante a Instalacao

1. Defina uma senha forte para o superusuario 'postgres'. Anote-a em local seguro.
2. Mantenha a porta padrao 5432.
3. Deixe o locale como Portuguese_Brazil (se disponível) ou UTF-8.
4. Conclua a instalacao.

4.4 Criacao do Banco de Dados e Tabelas

Abra o psql (Prompt do PostgreSQL) e execute os comandos abaixo.

```
# Conectar como superusuario psql -U postgres # Criar banco de dados  
CREATE DATABASE emprel_db; # Conectar ao banco recém-criado \c emprel_db
```

Em seguida, execute o script SQL completo (veja o Apendice ao final deste documento).

Dica: Salve o script SQL completo (disponível no final deste PDF) em um arquivo 'emprel_schema.sql' e execute com: psql -U postgres -d emprel_db -f emprel_schema.sql

5. Configuracao do Acesso Remoto

5.1 postgresql.conf

Localize o arquivo postgresql.conf. Caminhos comuns:

```
C:\Program Files\PostgreSQL\data\postgresql.conf C:\Program  
Files\PostgreSQL\data\pg_hba.conf
```

Edite postgresql.conf e altere:

```
# Altere esta linha: listen_addresses = 'localhost' # Para:  
listen_addresses = '*' # ou use o IP do servidor, ex: 192.168.1.10
```

5.2 pg_hba.conf

Edite pg_hba.conf para permitir conexoes da rede local. Adicione a linha abaixo (ajuste o IP conforme sua sub-rede):

```
# Tipo Banco Usuario Endereco Metodo host all all 192.168.1.0/24  
scram-sha-256
```

5.3 Firewall do Windows

1. Abra o Painel de Controle > Sistema e Seguranca > Firewall do Windows.
2. Clique em 'Configuracoes avancadas'.
3. Adicione uma regra de Entrada para a porta TCP 5432:

- Nome: PostgreSQL Local
- Protocolo: TCP
- Porta local: 5432
- Acao: Permitir conexao
- Perfil: Privado
- Aplicar a: Todos os programas

```
# Via PowerShell (executar como Administrador) New-NetFirewallRule  
-DisplayName "PostgreSQL" -Direction Inbound -Protocol TCP -LocalPort 5432  
-Action Allow -Profile Private
```

6. Compartilhamento da Pasta do Sistema (SMB)

6.1 Estrutura da Pasta Compartilhada

No servidor, organize a pasta do sistema da seguinte forma:

```
C:\Sistema_Emprel\ |-- sistema.exe (ou pythonw sistema.py) |-- script_sql\
| |-- emprel_schema.sql |-- logs\ # logs da aplicacao (opcional) |--
backup\ # backups locais antes do pg_dump |-- dados\ # arquivos JSON
antigos (somente leitura) |-- pdf\ # relatorios gerados
```

6.2 Criar Compartilhamento SMB

1. Clique com o botão direito na pasta C:\Sistema_Emprel.
2. Propriedades > Compartilhar > Compartilhamento avançado.
3. Marque 'Compartilhar esta pasta'.
4. Nome do compartilhamento: EMPREL
5. Permissões: conceda 'Leitura/Gravacao' ao grupo 'Todos' ou a usuarios especificos.

```
# Compartilhar via PowerShell New-SmbShare -Name EMPREL -Path
'C:\Sistema_Emprel' -FullAccess 'Todos'
```

6.3 Permissões NTFS

Altere as permissões NTFS para garantir que apenas usuarios autorizados tenham acesso.

```
# Conceder controle total a usuarios autorizados icacls
'C:\Sistema_Emprel' /grant 'DOMINIO\Usuarios:(OI)(CI)F' /T
```

Importante: Use (OI)(CI)F com cautela. Em redes corporativas, especifique usuarios em vez do grupo 'Todos'. O recursivo (/T) aplica a todas as subpastas.

7. Configuracao dos Clientes

7.1 Software Necessario

Em cada cliente (5 maquinas), instale:

- Python 3.8 ou superior (<https://www.python.org/downloads/>)
- Acesso a pasta compartilhada '\\\\SERVIDOR\\EMPREL'

7.2 Dependencias Python

No prompt de comando de cada cliente, instale as bibliotecas:

```
pip install reportlab pycopg2-binary
```

7.3 Configuracao da Conexao

No sistema, atualize as configuracoes de conexao para apontar para o servidor. Exemplo de conexao em Python (pycpg2):

```
import pycpg2 conn = pycpg2.connect( host='192.168.1.10', # IP do  
servidor port='5432', dbname='emprel_db', user='emprel_user',  
password='SuaSenhaSegura!' )
```

7.4 Mapeamento de Drive de Rede (Opcional)

Para facilitar o acesso, mapeie a pasta compartilhada como uma letra de drive.

```
# Mapear como letra X: net use X: \\SERVIDOR\\EMPREL /persistent:yes
```

Substitua 'SERVIDOR' pelo nome do computador ou IP do servidor.

8. Migracao dos Dados Existentes (JSON para PostgreSQL)

8.1 Preparacao

Antes de migrar, pare todos os clientes que estiverem usando o sistema. Faça um backup manual de todos os arquivos JSON.

8.2 Script de Migracao

Use o script Python abaixo para migrar os dados. Execute-o no servidor.

```
import json import psycopg2 # Conexao conn = psycopg2.connect(...) cur = conn.cursor() # Migrar usuarios with open('usuarios.json', 'r', encoding='utf-8') as f: usuarios = json.load(f) for u in usuarios: cur.execute('INSERT INTO usuarios ...', (...)) # Repetir para organizacoes, acoes, servicos, etc. conn.commit() conn.close()
```

8.3 Validacao

Apos a migracao, compare os totais para garantir que nenhum registro foi perdido:

```
SELECT COUNT(*) FROM usuarios; SELECT COUNT(*) FROM organizacoes; SELECT COUNT(*) FROM acoes;
```

Valide tambem os arquivos de exclusao (acoes_excluidas.json, cadastros_excluidos.json, servicos_excluidos.json) e migre os registros para a tabela historico_exclusoes.

9. Atalhos e Autostart nas Maquinas Clientes

9.1 Criar Atalho

1. Clique com o botao direito no arquivo executavel ou no pythonw.exe.
2. Selecione 'Enviar para > Area de trabalho (criar atalho)'.
3. Renomeie o atalho para 'Sistema EMPREL'.

9.2 Autostart por Pasta Inicializar

Copie o atalho para a pasta de Inicializar:

```
# Copiar atalho para Inicializar (executar no cliente) copy  
"%USERPROFILE%\Desktop\Sistema EMPREL.lnk"  
"%APPDATA%\Microsoft\Windows\Start Menu\Programs\Startup\"
```

9.3 Atalho na Pasta Compartilhada

Para garantir que o executavel sempre seja executado a partir da versao mais atual, crie um atalho na pasta compartilhada 'C:\Sistema_Emprel' e distribua esse atalho para os clientes. Os clientes acessam o caminho de rede.

10. Backups Automaticos

10.1 pg_dump

O utilitario pg_dump vem instalado com o PostgreSQL. Teste no servidor:

```
# Backup completo do banco "C:\Program Files\PostgreSQL\bin\pg_dump.exe"  
-U postgres -F c -b -v -f "C:\backup_emprel\backup_%DATE%.backup"  
emprel_db
```

10.2 Agendamento

Use o Agendador de Tarefas do Windows para rodar o backup diariamente.

```
# Script de backup (backup_emprel.bat) @echo off set  
DATA=%date:~6,4%-%date:~3,2%-%date:~0,2%_%time:~0,2%-%time:~3,2%  
"C:\Program Files\PostgreSQL\15\bin\pg_dump.exe" -U postgres -F c -b -v -f  
"C:\backup_emprel\backup_%DATA%.backup" emrel_db exit /b 0
```

10.3 Rotacao de Backups

Mantenha apenas os ultimos 30 backups. Exemplo de limpeza:

```
# No script batch, antes de gerar o novo backup: forfiles /p  
"C:\backup_emprel" /m *.backup /d -30 /c "cmd /c del @path"
```

Salve o backup em midia externa ou em outra unidade fisica periodicamente.

11. Seguranca e Manutencao

11.1 Usuario Dedicado do Banco

Nao utilize o superusuario 'postgres' na aplicacao. Crie um usuario com privilegios minimos:

```
# No psql CREATE USER emprel_user WITH PASSWORD 'SenhaSegura123!'; GRANT
CONNECT ON DATABASE emprel_db TO emprel_user; GRANT USAGE ON SCHEMA public
TO emprel_user; GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN
SCHEMA public TO emprel_user; GRANT USAGE, SELECT ON ALL SEQUENCES IN
SCHEMA public TO emprel_user;
```

11.2 Firewall e Rede

- Restrinja o pg_hba.conf apenas aos IPs da sua rede local (ex: 192.168.1.0/24).
- Nunca exponha a porta 5432 para a Internet.
- Se precisar de acesso externo, use VPN.

11.3 Manutencao Periodica

- Vacuum semanal (o PostgreSQL executa automaticamente, mas pode ser trigerrado manualmente):

```
VACUUM ANALYZE;
```

- Monitoramento espaco em disco e logs.
- Atualizacoes de sistema e PostgreSQL.
- Teste de restauracao de backup trimestralmente.

11.4 Politica de Senhas

- Senha do PostgreSQL com no mínimo 12 caracteres, letras maiusculas, minusculas, numeros e simbolos.
- Armazene a senha em gerenciador de senhas (nao em arquivos texto).
- Altere senhas a cada 90 dias.

Apendice: Script SQL Completo

Copie e cole o script abaixo no psql para criar todas as tabelas.

```
-- ===== -- Script:
emprel_schema.sql -- Sistema: EMPREL -- Banco: PostgreSQL --
===== --
----- -- Tabela: usuarios --
----- CREATE TABLE usuarios ( id SERIAL PRIMARY
KEY, nome VARCHAR(150) NOT NULL, usuario VARCHAR(80) NOT NULL UNIQUE,
senha VARCHAR(255) NOT NULL, nivel VARCHAR(30) NOT NULL DEFAULT
'operacional', ativo BOOLEAN NOT NULL DEFAULT TRUE, data_criacao TIMESTAMP
NOT NULL DEFAULT CURRENT_TIMESTAMP, data_atualizacao TIMESTAMP NOT NULL
DEFAULT CURRENT_TIMESTAMP ); -- ----- -- Tabela:
organizacoes -- ----- CREATE TABLE organizacoes (
id SERIAL PRIMARY KEY, razao_social VARCHAR(200) NOT NULL, nome_fantasia
VARCHAR(200), cnpj VARCHAR(18), endereco VARCHAR(300), telefone
VARCHAR(20), email VARCHAR(150), responsavel VARCHAR(150), ativo BOOLEAN
NOT NULL DEFAULT TRUE, data_cadastro TIMESTAMP NOT NULL DEFAULT
CURRENT_TIMESTAMP, data_atualizacao TIMESTAMP NOT NULL DEFAULT
CURRENT_TIMESTAMP ); -- ----- -- Tabela: acoes --
----- CREATE TABLE acoes ( id SERIAL PRIMARY KEY,
organizacao_id INTEGER NOT NULL REFERENCES organizacoes(id) ON DELETE
RESTRICT, numero_acao VARCHAR(50) NOT NULL, titulo VARCHAR(300) NOT NULL,
descricao TEXT, data_inicio DATE, data_fim DATE, status VARCHAR(30) NOT
NULL DEFAULT 'em_andamento', responsavel VARCHAR(150), observacoes TEXT,
ativo BOOLEAN NOT NULL DEFAULT TRUE, data_cadastro TIMESTAMP NOT NULL
DEFAULT CURRENT_TIMESTAMP, data_atualizacao TIMESTAMP NOT NULL DEFAULT
CURRENT_TIMESTAMP ); -- ----- -- Tabela: servicos
-- ----- CREATE TABLE servicos ( id SERIAL
PRIMARY KEY, organizacao_id INTEGER NOT NULL REFERENCES organizacoes(id)
ON DELETE RESTRICT, descricao VARCHAR(300) NOT NULL, valor NUMERIC(12,2)
DEFAULT 0.00, tipo VARCHAR(50), ativo BOOLEAN NOT NULL DEFAULT TRUE,
data_cadastro TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
data_atualizacao TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ); --
----- -- Tabela: cadastros --
----- CREATE TABLE cadastros ( id SERIAL PRIMARY
KEY, organizacao_id INTEGER NOT NULL REFERENCES organizacoes(id) ON DELETE
RESTRICT, nome VARCHAR(200) NOT NULL, documento VARCHAR(20) NOT NULL,
tipo_documento VARCHAR(10) NOT NULL DEFAULT 'CPF', funcao VARCHAR(100),
telefone VARCHAR(20), email VARCHAR(150), ativo BOOLEAN NOT NULL DEFAULT
TRUE, data_cadastro TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
data_atualizacao TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ); --
----- -- Tabela: historico_exclusoes --
----- CREATE TABLE historico_exclusoes ( id
SERIAL PRIMARY KEY, tabela_origem VARCHAR(50) NOT NULL, id_registro
INTEGER NOT NULL, dados_json JSONB NOT NULL, motivo VARCHAR(300),
usuario_id INTEGER REFERENCES usuarios(id), data_exclusao TIMESTAMP NOT
NULL DEFAULT CURRENT_TIMESTAMP ); -- ----- --
Tabela: entregas -- ----- CREATE TABLE entregas (
id SERIAL PRIMARY KEY, acao_id INTEGER NOT NULL REFERENCES acoes(id) ON
```

```

DELETE RESTRICT, organizacao_id INTEGER NOT NULL REFERENCES
organizacoes(id) ON DELETE RESTRICT, numero_documento VARCHAR(50),
data_entrega DATE NOT NULL DEFAULT CURRENT_DATE, responsavel VARCHAR(150),
observacoes TEXT, data_cadastro TIMESTAMP NOT NULL DEFAULT
CURRENT_TIMESTAMP ); -- ----- -- Tabela:
acao_ativa -- ----- CREATE TABLE acao_ativa ( id
SERIAL PRIMARY KEY, acao_id INTEGER NOT NULL UNIQUE REFERENCES acoes(id)
ON DELETE CASCADE, usuario_id INTEGER NOT NULL REFERENCES usuarios(id) ON
DELETE RESTRICT, data_abertura TIMESTAMP NOT NULL DEFAULT
CURRENT_TIMESTAMP, ip_origem VARCHAR(45), ativo BOOLEAN NOT NULL DEFAULT
TRUE ); -- ----- -- Tabela: logs --
----- CREATE TABLE logs ( id SERIAL PRIMARY KEY,
usuario_id INTEGER REFERENCES usuarios(id), acao VARCHAR(200) NOT NULL,
detalhes TEXT, ip_origem VARCHAR(45), data_log TIMESTAMP NOT NULL DEFAULT
CURRENT_TIMESTAMP ); -- ----- -- Indices
recomendados -- ----- CREATE INDEX
idx_acoes_organizacao ON acoes(organizacao_id); CREATE INDEX
idx_servicos_organizacao ON servicos(organizacao_id); CREATE INDEX
idx_cadastros_organizacao ON cadastros(organizacao_id); CREATE INDEX
idx_entregas_acao ON entregas(acao_id); CREATE INDEX
idx_entregas_organizacao ON entregas(organizacao_id); CREATE INDEX
idx_acao_ativa_acao ON acao_ativa(acao_id); CREATE INDEX idx_logs_usuario
ON logs(usuario_id); CREATE INDEX idx_logs_data ON logs(data_log); CREATE
INDEX idx_historico_exclusoes_tabela ON
historico_exclusoes(tabela_origem);

```

Apos criar as tabelas, gere o script de migracao dos dados JSON atuais para estas tabelas. Consulte um administrador de banco de dados se necessario.

Fim do documento.